

Interface Capability Brief

HTTP POST To RabbitMQ · Stakeholder overview · 2026-08-13

1. Executive summary

Accept an HTTP POST body on a PilotFish web-service path and publish that same payload to a RabbitMQ queue. Demo-only: no transformation, no auth, no dead-letter theater.

2. Who this is for

- Sources: any HTTP client (Web UI inject, curl, partner system) - Destinations: RabbitMQ queue demo.http.ingress (AMQP publish) - Demo vs production: Demo only (plain AMQP, demo/demo credentials, no TLS)

- Sources: any HTTP client (Web UI inject, curl, partner system)
- Destinations: RabbitMQ queue demo.http.ingress (AMQP publish)
- Demo vs production: Demo only (plain AMQP, demo/demo credentials, no TLS)

3. What the interface can do

Capabilities below are drawn from the working design and, when present, the published CapabilityStatement — not from aspirational roadmap items.

- Transport: HTTP POST listener com.pilotfish.eip.modules.http.HttpPostListener
- Path: /eip/http-post/ingress (RequestPath ingress)
- Format: opaque body (JSON sample provided; any bytes accepted)
- Identity: none required
- Samples path: samples/*.json

Integration highlights

Stage	Module / mechanism	Notes
Listener	com.pilotfish.eip.modules.http.HttpPostListener	Synchronous=false (playbook §1.4 — queue ingest is fire-and-forget)
FormatProfile	Relay	Pass-through (body is not XML)
Router	com.pilotfish.eip.modules.internal.NullRoutingModule	Single target; do not use XPath (JSON is not XML)
Transport	com.pilotfish.eip.modules.messaging.rabbitmq.RabbitMQTransport	Host and Port \$\$RABBITMQ_HOSTS, VirtualHost=/ (playbook §1.4 — not URI)

4. Honest boundaries (not claimed yet)

- Optional Basic auth on the HTTP path
- Optional topic exchange / routing-key theater

5. What you can verify in a walkthrough

Use these as checklist items when reviewing the demo with business stakeholders. They describe outcomes, not module names.

- Exercise route “1 - HTTP POST To RabbitMQ” from the Web UI or documented smoke path
- Confirm outputs land in the documented folders / queues and appear in the Web UI status views

6. How it works (plain language)

7. Validation

- What is checked: HTTP POST reaches the listener path - What is NOT checked: JSON schema, content-type, AMQP properties beyond ContentType - Does failure block outbound? Listener/transport exceptions fail the transaction (logs/eip.log)

6. State & idempotency

- Status model: queue depth only - When state advances: publish succeeds - Dedup keys: none (demo will enqueue duplicates) - Retry / poison: not implemented (documented)

8. Dual-write / side effects

- Single write: RabbitMQ publish - Compensation: none

3. Inbound contract

- Transport: HTTP POST listener `com.pilotfish.eip.modules.http.HttpPostListener` - Path: `/eip/http-post/ingress` (RequestPath ingress) - Format: opaque body (JSON sample provided; any bytes accepted) - Identity: none required - Samples path: `samples/*.json`

Routes in this interface

Each route is a configured PilotFish flow. Technical wiring appears in the Route Diagrams PDF; here is the stakeholder reading.

1 - HTTP POST To RabbitMQ: runs steps such as HTTP POST Ingress, Publish To Queue.

7. Security & trust posture

- Credentials and passwords in this package are demo-only — not production secrets
- Risk noted in design: Med — Docs are 26R1.11; runtime is `pilotfish-eip:23R1` — Both modules exist in 23R1 jars (`modules-http-23R1-SNAPSHOT.jar`, `modules-rabbitmq-23R1-SNAPSHOT.jar`)
- Risk noted in design: Low — Guest AMQP user is remote-disabled on modern RabbitMQ — Demo user `demo/demo` via `RABBITMQ_DEFAULT_USER`
- Risk noted in design: Low — HTTP listener is async — Caller gets an immediate POST ack; queue peek confirms publish
- Risk noted in design: Low — No TLS / no auth on HTTP — Demo only

8. Seeing it work

- - Compose project: `http-post-to-rabbitmq` - Ports: EIP 8134, Web UI 8135, RabbitMQ AMQP 5673, RabbitMQ management 15673 - Local Web UI: `http://localhost:8135/` - LAN Web UI: `http://192.168.68.62:8135/` - EIP: `http://localhost:8134/eip/` - POST URL: `http://localhost:8134/eip/http-post/ingress` - RabbitMQ management: `http://localhost:15673/` (`demo/demo`) - Compose profile stage = Web UI only; full = RabbitMQ + EIP + Web UI - Cold start ~60–90s for EIP after RabbitMQ is healthy
- - Local: `http://localhost:8135/`
- - EIP: `http://localhost:8134/eip/`
- - POST path: `http://localhost:8134/eip/http-post/ingress`
- - RabbitMQ management: `http://localhost:15673/` (user `demo` / `demo`)
- `docker compose --profile full up -d --build`
- `docker compose --profile stage up -d --build`

9. Related technical documents

-
- DESIGN.md — working engineering specification for this interface
 - README.md — how to run, ports, and smoke commands
 - HTTP_POST_To_RabbitMQ_V2_Route_Diagrams.pdf — detailed PilotFish route wiring diagrams
 - This Capability Brief — shareable stakeholder summary (regenerate after design changes)

This document is generated from the interface sources listed above. If design and runtime diverge, believe the smoke-tested runtime and update DESIGN.md.